Road Junction Graph Using Degree Calculation

Problem Statement 1: Original Program

1. Objective

A road network is modelled as an undirected graph: each junction is a vertex and each road is an edge. The program stores the network in an adjacency list, displays it, computes the degree of every junction (the number of roads meeting there), and lists the junctions whose degree is at least a user-given threshold K.

2. Main Operations

- Read the number of junctions and roads.
- Build an undirected graph; each road u-v is stored in both adjacency lists.
- Display the adjacency list.
- Compute and display the degree of each junction.
- Read K and display junctions with degree greater than or equal to K.

3. Design Notes

- **Adjacency list:** a vector of vectors, indexed 1 to n, so each junction's neighbours are stored directly.
- **Degree:** for a simple undirected graph, the degree of a vertex is simply the size of its adjacency list.
- **Input validation:** roads referring to a junction outside 1 to n, or self-loops (u = v), are rejected with a message.
- **Complexity:** building the list takes O(M) time and O(N + M) space; computing degrees takes O(N); filtering takes O(N).

4. Source Code (road_junction.cpp)

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;

class RoadNetwork {
    int junctions;
    vector<vector<int>> adj;   // adjacency list (1-indexed)

public:
    explicit RoadNetwork(int n) : junctions(n), adj(n + 1) {}

    // Undirected: store the road in both directions
    bool addRoad(int u, int v) {
        if (u < 1 || u > junctions || v < 1 || v > junctions) return false;
        if (u == v) return false;               // ignore self-loops
        adj[u].push_back(v);
        adj[v].push_back(u);
        return true;
    }

    void displayGraph() const {
        cout << "\nAdjacency List of the Road Network:\n";
        for (int i = 1; i <= junctions; i++) {
            cout << "Junction " << setw(2) << i << " -> ";
            if (adj[i].empty()) {
                cout << "(no roads)";
            } else {
                for (size_t j = 0; j < adj[i].size(); j++) {
                    cout << adj[i][j];
                    if (j + 1 < adj[i].size()) cout << ", ";
                }
            }
            cout << "\n";
        }
    }

    int degree(int v) const { return static_cast<int>(adj[v].size()); }

    void displayDegrees() const {
        cout << "\nDegree of Each Junction:\n";
        cout << setw(10) << "Junction" << setw(10) << "Degree" << "\n";
        cout << string(20, '-') << "\n";
        for (int i = 1; i <= junctions; i++)
            cout << setw(10) << i << setw(10) << degree(i) << "\n";
    }

    void displayHighDegree(int k) const {
        cout << "\nJunctions with degree >= " << k << ":\n";
        bool found = false;
        for (int i = 1; i <= junctions; i++) {
            if (degree(i) >= k) {
                cout << "  Junction " << i << " (degree " << degree(i) << ")\n";
                found = true;
            }
        }
        if (!found) cout << "  No junction satisfies this condition.\n";
    }
};

int main() {
    int n, m;
    cout << "Enter the number of junctions: ";
    cin >> n;
    if (n <= 0) {
        cout << "Number of junctions must be positive.\n";
        return 1;
    }

    cout << "Enter the number of roads: ";
```

```
    cin >> m;
    if (m < 0) {
        cout << "Number of roads cannot be negative.\n";
        return 1;
    }

    RoadNetwork network(n);

    cout << "Enter each road as two junction numbers (u v):\n";
    int accepted = 0;
    for (int i = 0; i < m; i++) {
        int u, v;
        cout << "Road " << i + 1 << ": ";
        cin >> u >> v;
        if (network.addRoad(u, v)) {
            accepted++;
        } else {
            cout << "  Invalid road ignored (check junction numbers 1-" << n
                 << " and that u != v).\n";
        }
    }
    cout << "\n" << accepted << " road(s) added to the network.\n";

    network.displayGraph();
    network.displayDegrees();

    int k;
    cout << "\nEnter threshold K to find highly connected junctions: ";
    cin >> k;
    network.displayHighDegree(k);

    return 0;
}
```

5. Compilation and Execution

```
g++ -std=c++17 -Wall -Wextra -o road_junction road_junction.cpp
./road_junction
```

6. Sample Run

Input: 6 junctions, 7 roads (one of them, 3 3, is a self-loop and is rejected), and threshold K = 3.

```
Enter the number of junctions: 6
Enter the number of roads: 7
Enter each road as two junction numbers (u v):
Road 1: 1 2
Road 2: 1 3
Road 3: 1 4
Road 4: 2 3
Road 5: 3 4
Road 6: 4 5
Road 7: 3 3
  Invalid road ignored (check junction numbers 1-6 and that u != v).

6 road(s) added to the network.

Adjacency List of the Road Network:
Junction  1 -> 2, 3, 4
Junction  2 -> 1, 3
Junction  3 -> 1, 2, 4
Junction  4 -> 1, 3, 5
Junction  5 -> 4
Junction  6 -> (no roads)

Degree of Each Junction:
  Junction    Degree
--------------------
         1         3
         2         2
         3         3
```

```
        4         3
        5         1
        6         0

Enter threshold K to find highly connected junctions: 3
Junctions with degree >= 3:
  Junction 1 (degree 3)
  Junction 3 (degree 3)
  Junction 4 (degree 3)
```

7. Interpretation

Junctions 1, 3 and 4 each have three connecting roads, so they are the highly connected junctions for K = 3. Junction 6 has no roads at all and therefore degree 0.